

Esercitazione GUIDATA: Processi e Non Determinismo con fork()

Obiettivo

Capire il concetto di **non determinismo** e la **duplicazione delle variabili** nei processi creati con `fork()` in C.

Parte 1: Comprensione della duplicazione delle variabili

La funzione `fork()` crea un nuovo processo duplicando quello corrente. Restituisce:

- **Un valore positivo** nel processo padre, che rappresenta il PID del processo figlio.
- **Zero** nel processo figlio.
- **Un valore negativo** se la creazione del processo fallisce.
-

1. Creare un file `duplication.c` e scrivere il seguente codice:

```
#include <stdio.h>
#include <unistd.h>

int main() {
    int x = 10;
    printf("[PRIMA di aver chiamato FORK] PID: %d, x = %d\n", getpid(), x);

    pid_t pid = fork();

    printf("[DOPO aver chiamato FORK] PID: %d, x = %d\n", getpid(), x);

    if (pid == 0) {
        // Processo figlio
        x += 5;
        printf("[FIGLIO] PID: %d, x = %d\n", getpid(), x);
    } else {
        // Processo padre
        x -= 5;
        printf("[PADRE] PID: %d, x = %d\n", getpid(), x);
    }

    printf("[PADRE/FIGLIO?] PID: %d, x = %d\n", getpid(), x);
    return 0;
}
```

2. Domande di riflessione:

- Quanti processi vengono creati dopo la `fork()` ?
- Quali valori vengono stampati da padre e figlio?
- Il valore di `x` viene condiviso tra i processi?

Parte 1.1: Esercizio sulla copia delle variabili

1. Creare un file `copy_variables.c` e scrivere il seguente codice:

```
#include <stdio.h>
#include <unistd.h>

int main() {
    int a = 20;
    int b = 30;
    printf("[PRIMA DEL FORK] PID: %d, a = %d, b = %d\n", getpid(), a, b);

    pid_t pid = fork();
```

```

if (pid == 0) {
    // Processo figlio modifica le variabili
    a += 10;
    b -= 10;
    printf("[FIGLIO] PID: %d, a = %d, b = %d\n", getpid(), a, b);
} else {
    // Processo padre modifica le variabili
    a -= 10;
    b += 10;
    printf("[PADRE] PID: %d, a = %d, b = %d\n", getpid(), a, b);
}

return 0;
}

```

2. Domande di riflessione:

- I valori delle variabili `a` e `b` cambiano nello stesso modo per padre e figlio?
- Perché le modifiche fatte dal padre non influenzano il figlio e viceversa?

Parte 2: Osservazione del non determinismo

1. Creare un file `nondeterminism.c` e scrivere il seguente codice:

```

#include <stdio.h>
#include <unistd.h>

int main() {
    printf("[PRIMA DEL FORK] Processo PID: %d\n", getpid());

    pid_t pid = fork();

    printf("[DOPO FORK] Processo PID: %d, PID del padre: %d\n", getpid(), getppid());
    return 0;
}

```

2. Eseguire più volte il programma e rispondere:

- L'ordine delle stampe è sempre lo stesso?
- Perché il risultato cambia?
- Cosa succede se si aggiunge `sleep(1);` nel figlio prima della stampa?

Parte 3: Creazione di più processi

1. Creare un file `multiple_forks.c` e scrivere il codice:

```

#include <stdio.h>
#include <unistd.h>

int main() {
    printf("[INIZIO] PID: %d\n", getpid());
    fork();
    fork();
    printf("[DOPO FORK] PID: %d\n", getpid());
    return 0;
}

```

2. Domande:

- Quanti processi vengono creati?
- Quante stampe avvengono?
- Come si può controllare il numero di processi generati?

Conclusione

- La `fork()` crea processi indipendenti che **non condividono le variabili locali**.
- L'ordine di esecuzione dei processi non è deterministico, dipende dal sistema operativo.
- Più `fork()` possono creare una **crescita esponenziale** dei processi.